# GRPO with QLoRA for Verified CISC-to-RISC Assembly Transpilation

**Mid-point Report**

**Enzo Shiraishi Nunes da Silva**
Mohammed Bin Zayed University of Artificial Intelligence
enzo.shiraishi@mbzuai.ac.ae
*PhD in Computer Science*

**Sergei Pasynkov**
Mohammed Bin Zayed University of Artificial Intelligence
sergei.pasynkov@mbzuai.ac.ae
*PhD in Computer Science*

**Zayed Khaled Al Tamimi**
Mohammed Bin Zayed University of Artificial Intelligence
zayed.altamimi@mbzuai.ac.ae
*PhD in Computer Science*

## 1 Introduction

Translating assembly from one processor family to another, for example from x86 to ARM, is a practical need when software must move to new hardware. It is difficult to do by hand and difficult to guarantee. Guaranteed Guess (GG) addresses this with a language model that proposes a translation and software tests that accept or reject it [1]. In this report, we study whether the same tests can also be used to train the model, with reinforcement learning, so that it produces more translations that pass.

Our earlier proposal applied DAPO to vision-language-action (VLA) models, which produce robot actions from images and instructions [9]. We changed the topic for two reasons. First, research on VLAs is currently centered on deployment cost, namely latency and memory [4, 5]. The existing reinforcement learning work for VLAs reports task success and relies on robot simulation to produce each training sample, and the feedback from these simulations is often sparse [6–8]. This is costly under limited compute. Second, translating assembly poses the same research question in a setting that our compute can support. The output can be checked exactly by running tests, one attempt is a single generation followed by a test run, and the model is small enough for parameter-efficient training on one GPU [1]. The problem is also practically relevant, because the translated code of this approach runs faster than Apple's Rosetta 2 [1].

The method we study is Group Relative Policy Optimization (GRPO) with a test-based reward, combined with Quantized Low-Rank Adaptation (QLoRA) to reduce memory use [11, 21]. GRPO learns only from problems on which some attempts pass and others fail, so it needs a starting model that already solves part of the problems. The released GG checkpoint passes 44% of the HumanEval-C problems at O2 in our evaluation (Section 3). We therefore start from it instead of repeating its supervised fine-tuning, which saves compute.

The rest of this report contains a short review of related work (Section 2), the baseline we evaluate and its results (Section 3), the progress and plan for our method (Section 4), and the remaining work with its distribution among the authors (Section 5).

Preprint.

## 2 Related Work

**Reinforcement learning with verifiable rewards.** Reinforcement learning with verifiable rewards (RLVR) uses a program to check the answer instead of a learned reward model [10]. GRPO suits this setting because it compares several attempts at the same problem and does not require a separate model to predict their results [11]. The approach has given strong results on verifiable tasks such as mathematics and coding [12], and GRPO with binary rewards has been shown to increase the success probability of the starting model [13].

**RLVR for code.** Code is a natural fit for RLVR because tests give an exact and inexpensive result. RLEF trains code models on the results of running their own code [14], and VeRPO uses partial test results as a denser reward [15]. Recent works apply the same idea to low-level code, where the output can be executed or formally verified. SuperCoder uses reinforcement learning to produce faster assembly programs, rewarding both correctness and speed [16]. LLM-VeriOpt trains a small model with GRPO, using a formal verifier as the reward, to optimize compiler code [17]. AutoDecompiler uses compilation and test feedback to train a model that converts binaries back into source code [18], and TRAVEL applies reinforcement learning with execution feedback to translate C code into Rust [19]. These works show that RLVR can improve code, assembly, and translation. To our knowledge, none of them applies it to translating CISC assembly into RISC assembly.

**Assembly translation.** Guess & Sketch combines guesses from a language model with a solver that checks whether the translation is equivalent [3]. CRT is a lightweight language model that translates x86 assembly into ARM and RISC-V [2]. GG fine-tunes small pre-trained code models (DeepSeek-Coder 1.3B and Qwen2.5-Coder 0.5B and 1.5B) on about 1.3 million C programs from AnghaBench and The Stack v2, compiled to both instruction sets. It generates candidate translations and checks them with software tests. It is evaluated on HumanEval [22], a set of programming problems whose C versions are translated and then executed, and on BringupBench, a set of larger programs for embedded systems [1]. In GG, the tests only accept or reject the output of the model. We use the same tests as a training signal.

**Parameter-efficient training.** Low-Rank Adaptation (LoRA) trains a small number of added weights instead of the whole model [20]. QLoRA does the same on top of a base model stored in 4-bit precision, which further reduces memory use [21]. We use both so that training fits on a single GPU.

## 3 Baseline

Our baseline is the released GG model for ARMv8 at optimization level O2, which is Qwen2.5-Coder-1.5B-Instruct fine-tuned on the GG data [1]. We do not train it.

**Evaluation.** We evaluate on the 164 HumanEval-C problems at O2. The input is the x86 assembly of the official GG benchmark. The tests are the authors' own C test programs, which they released on Hugging Face (`asmbench_source`). A problem counts as solved if the translation passes all of its tests, and we report the fraction of solved problems (pass@1). Our reward harness checks each translation: the predicted ARM assembly is the only definition of the function in the test program, which is assembled, linked, and run in a sandbox without network access, with limits of 10 s wall time and 5 s CPU time. As a check of the harness, the official reference ARM assembly passes all 164 problems, and an empty prediction fails all 164. We use the prompt and the answer extraction of the authors' evaluation script, and beam search with 8 beams, as stated in the paper.

**Results.** Table 1 gives the results. Our first run solves 72 of the 164 problems (43.9%), 2 problems fewer than the 74 (45.1%) reported by the authors. Further runs, listed in the table, test the remaining differences in the evaluation settings.

**Sources of the difference.** Most of the original difference came from the tests. We first used the HumanEval-C tests of LLM4Decompile [24]. With them, the same translations solve 68 problems, and the official reference assembly fails 6, because for these 6 problems the official assembly was built from a different C function. With the authors' tests, the reference passes all 164 problems, and

Table 1: Problems solved (pass@1) by the released GG O2 model. HumanEval-C has 164 problems. BringUp has 66 programs, of which 65 can be built; percentages are out of 65, as in the paper.

| Source | Decoding and limits | HumanEval-C O2 | BringUp O2 |
|---|---|---|---|
| GG paper [1] | beam 8 | 74 (45.1%) | 10 (15.4%) |
| Ours | beam 8, 16,000 new tokens | 72 (43.9%) | – |
| Ours | beam 8, 32,000 tokens, transformers 4.50 | [running] | [running] |
| Ours | beam sampling, authors' script as released | [queued] | – |
| Ours | greedy, authors' BringUp script as released | – | [running] |

the two test sets give the same result on the other 160. The remaining 2 problems may come from three differences, which we test in separate runs:

- *Length limit.* Our first run stops after 16,000 new tokens, while the authors' script allows 32,000 tokens minus the prompt. 7 of our 164 outputs reached the limit, and all 7 failed.
- *Decoding.* The paper states that sampling was disabled, but the authors' script sets only the number of beams. The model's configuration file then turns sampling on (temperature 0.7, top-$k$ 20, top-$p$ 0.8), so the script runs beam sampling.
- *Software.* Our first run used transformers 5.20 with PyTorch's attention; the model was released with transformers 4.50, and the authors' script uses FlashAttention-2.

The compiler is not a cause: our toolchain reproduces the official data exactly (Section 4). We have asked the authors which decoding produced the published result.

## 4 Proposed Method

**Method.** We apply GRPO to the GG checkpoint. For each problem, the model generates a group of $G = 8$ translations. Translation $i$ receives a reward $r_i = 1$ if it passes all tests and $r_i = 0$ otherwise. Its advantage is its reward relative to the rest of the group,

$$\hat{A}_i = \frac{r_i - \text{mean}(r_1, \ldots, r_G)}{\text{std}(r_1, \ldots, r_G)}, \tag{1}$$

and the model is updated to make translations with a positive advantage more likely. A group in which all translations pass, or all fail, gives no update. We use the GRPO implementation in the TRL library without a KL penalty, and we write the reward function, the data preparation, and the evaluation code ourselves. The two training variants differ only in the base weights: LoRA on a 16-bit (bfloat16) base model, and QLoRA on a 4-bit (NF4) base model [20, 21]. Both use the same settings, and both are compared with the baseline of Section 3 under the same evaluation.

**Data.** The training problems come from the C version of MBPP [23] prepared by LLM4Decompile [24], which includes assertion tests. We compile each function to x86 and ARM assembly at O2 with the GG toolchain. Of the 973 problems that compile, we keep the 952 whose reference ARM assembly passes its own tests. The other 21 have faulty tests, for example tests that compare strings by address. HumanEval-C is used only for evaluation, and BringUp at O2 is a second evaluation for the final models. We start with MBPP-C only and will add ExeBench only if more training problems are needed.

**Settings.** The following settings are fixed: a group size of 8, sampling at temperature 1 without top-$k$ or top-$p$, no KL penalty, translations cut by the length limit excluded from the loss, LoRA on all linear layers, generation with vLLM [25] inside the training process, and bfloat16 training. The following settings will be fixed before the first run: the LoRA rank and scaling, the learning rate, the batch size, the number of steps (we expect 100 to 300), the maximum translation length, and the loss aggregation (per sequence as in GRPO, or per token as in DAPO [9]). For the length limit, the reference ARM assembly of the training problems has a median length of 717 tokens, and 97 of the 952 references exceed 2,048 tokens.

**Progress.** *Toolchain.* The official GG data was produced with Xcode 14.3 and the macOS 13.3 SDK, and our Mac has a newer toolchain that produces different assembly. We therefore built a macOS 13.6 virtual machine with Xcode 14.3.1 (Apple clang 14.0.3, SDK 13.3). Compiling the authors' C sources in it reproduces all 328 official HumanEval-C files (x86 and ARM, O0 and O2) and 131 of the 132 BringUp files byte for byte. All our training and test programs are built in this machine.

*Reward harness.* The harness runs as a scoring service inside the virtual machine, because the GG data is ARM assembly for macOS and can only be assembled and run on Apple Silicon. The same service gives the training reward and the evaluation results. Besides the HumanEval-C checks of Section 3, the reference assembly of all 952 training problems passes. For BringUp we use the authors' test procedure, with which the official reference assembly passes 65 of the 66 programs. The remaining program cannot be built, which is consistent with the paper's BringUp results, all of which are multiples of 1/65.

*Training.* The training machine is a Linux server with one NVIDIA RTX PRO 6000 Blackwell GPU (96 GB), which reaches the scoring service through a single network tunnel. This differs from our earlier assumption of up to two GPUs. A second machine with a 16 GB GPU runs evaluations. The TRL configuration and the reward function are written and tested against the scoring service. Training starts once the remaining settings are fixed.

**Risks.**

- *QLoRA with fast generation.* With a 4-bit base model, TRL adds the LoRA weights into the 4-bit weights of the generation engine at every update and removes them afterwards. We have not verified that this works, or that rounding does not slowly change the base weights. We will test it on a short run before any long training. If it fails, both variants will generate with the training model itself, so that they still differ only in the base weights.
- *Size of the test set.* With 164 problems, one problem is 0.6 percentage points, so small differences between models cannot be distinguished from noise. We will report confidence intervals and compare models problem by problem with a paired test.
- *Overlap between datasets.* We have not yet checked whether MBPP-C problems overlap with HumanEval-C or with the GG training data. We will check this and remove overlapping problems before training.
- *Throughput.* The scoring service runs on a single Mac, and one check takes 0.1 to 0.5 s. Its throughput limits the number of training steps.

## 5 Remaining Work and Distribution

| Dates | Work |
|---|---|
| Oct. 24–30 | Complete the baseline runs of Table 1. Measure the baseline with sampling at temperature 1, the setting used during GRPO, on HumanEval-C and on the training problems. Check the overlap between datasets. Fix the training settings and run short LoRA and QLoRA tests. |
| Oct. 31–Nov. 13 | Train the LoRA and QLoRA variants with identical settings. |
| Nov. 14–27 | Evaluate both variants and the baselines on HumanEval-C and BringUp at O2. Train the third baseline. Run the ablation and the parameter study. |
| Nov. 28–Dec. 8 | Analyze the results and the types of failure, prepare the final report, and release the code. |

**Baselines.** The final report will compare GRPO with QLoRA against three baselines: (1) the released GG model; (2) GRPO with LoRA on the 16-bit model, which shows the cost of the 4-bit base; and (3) rejection-sampling fine-tuning [26], in which the GG model translates each training problem several times and is fine-tuned with LoRA on the translations that pass. The third baseline uses the same tests as GRPO, but without reinforcement learning. `[third baseline to confirm]`

**Ablation and parameter study.** [to confirm] As the ablation, we compare loss aggregation per sequence (GRPO) with aggregation per token (DAPO). As the parameter study, we vary the LoRA rank of the QLoRA variant.

**Workload distribution.**

| Work | Member |
|---|---|
| Training: settings, short tests, LoRA and QLoRA runs | [member] |
| Evaluation and baselines: HumanEval-C, BringUp, third baseline | [member] |
| Data and analysis: overlap check, ablation, parameter study, failure types | [member] |
| Final report and code release | All members |

## References


[1] A. Heakl, S. Hashmi, C. Abi, C. Lee, and A. Mahmoud. Guaranteed Guess: A Language Modeling Approach for CISC-to-RISC Transpilation with Testing Guarantees. arXiv:2506.14606, 2025. https://arxiv.org/abs/2506.14606.

[2] A. Heakl, C. Abi, R. Hossam, and A. Mahmoud. From CISC to RISC: Language-Model Guided Assembly Transpilation. arXiv:2411.16341, 2024. https://arxiv.org/abs/2411.16341.

[3] C. Lee, A. Mahmoud, M. Kurek, S. Campanoni, D. Brooks, S. Chong, G.-Y. Wei, and A. M. Rush. Guess & Sketch: Language Model Guided Transpilation. arXiv:2309.14396, 2023. https://arxiv.org/abs/2309.14396.

[4] W. Guan, Q. Hu, A. Li, and J. Cheng. Efficient Vision-Language-Action Models for Embodied Manipulation: A Systematic Survey. arXiv:2510.17111, 2025. https://arxiv.org/abs/2510.17111.

[5] Z. Yu et al. A Survey on Efficient Vision-Language-Action Models. arXiv:2510.24795, 2025. https://arxiv.org/abs/2510.24795.

[6] H. Li et al. SimpleVLA-RL: Scaling VLA Training via Reinforcement Learning. arXiv:2509.09674, 2025. https://arxiv.org/abs/2509.09674.

[7] G. Lu et al. VLA-RL: Towards Masterful and General Robotic Manipulation with Scalable Reinforcement Learning. arXiv:2505.18719, 2025. https://arxiv.org/abs/2505.18719.

[8] H. Zang et al. RLinf-VLA: A Unified and Efficient Framework for Reinforcement Learning of Vision-Language-Action Models. arXiv:2510.06710, 2025. https://arxiv.org/abs/2510.06710.

[9] Q. Yu et al. DAPO: An Open-Source LLM Reinforcement Learning System at Scale. arXiv:2503.14476, 2025. https://arxiv.org/abs/2503.14476.

[10] N. Lambert et al. Tulu 3: Pushing Frontiers in Open Language Model Post-Training. arXiv:2411.15124, 2024. https://arxiv.org/abs/2411.15124.

[11] Z. Shao et al. DeepSeekMath: Pushing the Limits of Mathematical Reasoning in Open Language Models. arXiv:2402.03300, 2024. https://arxiv.org/abs/2402.03300.

[12] DeepSeek-AI. DeepSeek-R1: Incentivizing Reasoning Capability in LLMs via Reinforcement Learning. Nature, 645:633–638, 2025; arXiv:2501.12948. https://arxiv.org/abs/2501.12948.

[13] Y. Mroueh. Reinforcement Learning with Verifiable Rewards: GRPO's Effective Loss, Dynamics, and Success Amplification. arXiv:2503.06639, 2025. https://arxiv.org/abs/2503.06639.

[14] J. Gehring, K. Zheng, J. Copet, V. Mella, Q. Carbonneaux, T. Cohen, and G. Synnaeve. RLEF: Grounding Code LLMs in Execution Feedback with Reinforcement Learning. arXiv:2410.02089, 2024. https://arxiv.org/abs/2410.02089.

[15] L. Wang et al. Beyond Binary: Turning Partial Success into Dense Verifiable Rewards for Reinforcement Learning in Code Generation. arXiv:2601.03525, 2026. https://arxiv.org/abs/2601.03525.

[16] A. Wei, T. Suresh, H. Tan, Y. Xu, G. Singh, K. Wang, and A. Aiken. SuperCoder: Assembly Program Superoptimization with Large Language Models. arXiv:2505.11480, 2025. https://arxiv.org/abs/2505.11480.

[17] X. Fang, J. Kang, R. C. Rocha, S. Ainsworth, and L. Mukhanov. LLM-VeriOpt: Verification-Guided Reinforcement Learning for LLM-Based Compiler Optimization. IEEE/ACM International Symposium on Code Generation and Optimization (CGO), 2026. https://2026.cgo.org/details/cgo-2026-papers/37/LLM-VeriOpt-Verification-Guided-Reinforcement-Learning-for-LLM-Based-Compiler-Optimi.

[18] P. Liu et al. Binary Decompilation LLM with Feedback-Driven Multi-Turn Refinement. arXiv:2606.16162, 2026. https://arxiv.org/abs/2606.16162.

[19] F. Luo, J. Liu, C. Gao, J. Feng, and K. Liu. Towards Reliable C-to-Rust Translation with Rule-Guided Reasoning and Reinforcement Learning. arXiv:2607.19966, 2026. https://arxiv.org/abs/2607.19966.

[20] E. J. Hu et al. LoRA: Low-Rank Adaptation of Large Language Models. International Conference on Learning Representations (ICLR), 2022; arXiv:2106.09685. https://arxiv.org/abs/2106.09685.

[21] T. Dettmers, A. Pagnoni, A. Holtzman, and L. Zettlemoyer. QLoRA: Efficient Finetuning of Quantized LLMs. Advances in Neural Information Processing Systems (NeurIPS), 2023; arXiv:2305.14314. https://arxiv.org/abs/2305.14314.

[22] M. Chen et al. Evaluating Large Language Models Trained on Code. arXiv:2107.03374, 2021. https://arxiv.org/abs/2107.03374.

[23] J. Austin et al. Program Synthesis with Large Language Models. arXiv:2108.07732, 2021. https://arxiv.org/abs/2108.07732.

[24] H. Tan et al. LLM4Decompile: Decompiling Binary Code with Large Language Models. Conference on Empirical Methods in Natural Language Processing (EMNLP), 2024; arXiv:2403.05286. https://arxiv.org/abs/2403.05286.

[25] W. Kwon et al. Efficient Memory Management for Large Language Model Serving with PagedAttention. ACM Symposium on Operating Systems Principles (SOSP), 2023; arXiv:2309.06180. https://arxiv.org/abs/2309.06180.

[26] Z. Yuan et al. Scaling Relationship on Learning Mathematical Reasoning with Large Language Models. arXiv:2308.01825, 2023. https://arxiv.org/abs/2308.01825.